Agent Smith: A Specification Language for Instantiating Purpose-Directed Agents, with a Compiler That Instantiates Them Honouring Four Invariants

Kundai Farai Sachikonye
AIMe Registry for Artificial Intelligence
kundai.sachikonye@bitspark.com

July 22, 2026

Abstract

A game designer specifies a non-player character not by writing its every motion but by saying what it *is* and what it is *for*: an iron smith who forges, contracts, and sources ore; a baker who bakes because he must. We give a language that lets any user, application, or module specify an agent in exactly this way—by declaring what the agent is for, not how each act is performed—and a compiler that turns the specification into a running agent. The language is **Agent Smith**; every script written in it *is* an Agent Smith, a complete specification of one agent. We fix the language's objects as those of the underlying theory of purpose-directed agents: a self-graph carrying a conserved identity, a purpose that is the attractor of a declared drive, a finite family of scenes the agent may act in, a coherence condition it must not break, and a bounded attention budget. We give Agent Smith a grammar, a type system, and a small-step lowering semantics into agent objects, and we prove the compiler's central guarantee: *every well-typed Agent Smith compiles to an agent that honours, by construction, four runtime invariants—conserved identity, a never-resetting committed count, search-not-fetch world access, and construction/commitment phase exclusion—and no ill-typed script is instantiated.* We prove the language is *mechanism-agnostic*: it specifies the agent's purpose and the scenes that serve it, and delegates every domain behaviour to a constructor-supplied hook, so that Agent Smith never contains the competence of any domain (how to forge, how to bake, how to solve) but only the means to instantiate an agent that pursues one. We prove a society of agents is specified by the same language one level up, so any configuration of agents—an ensemble instantiated and used as the specifier chooses—is itself an Agent Smith. Because the agent an Agent Smith instantiates is, by the underlying theory, indifferent between "living a life" and "executing a task"—the two are one object under two readings of where its purpose rests—the language instantiates game characters and task-solving agents by the same constructs. The compiler is the instantiator; the specifier declares intent; the agent does the rest. Interpretive readings—of a script as a character, of a society as a population—are confined to remarks on which no theorem depends.

Keywords: specification language; agent instantiation; domain-specific language; typed compiler; lowering semantics; runtime invariant; purpose-directed agent; non-player character; mechanism agnosticism; society of agents.

1 Introduction

1.1 Specifying an agent the way one specifies an NPC

A designer who wants an iron smith in a game does not write the smith's every hammer-blow. They say what the smith *is*—a smith, with a workshop, a standing in the town—and what the smith is *for*: to forge, to take contracts, to keep itself supplied with ore. The rest—which of these the smith does at a given moment, how it forges a particular blade—follows from what it is for and from the competence of forging, which the designer does not re-derive. A convincing character is *specified by its purpose*, and its conduct is the pursuit of that purpose through whatever activities serve it.

This paper gives a language for specifying agents in exactly this way, and a compiler that instantiates them. The language is **Agent Smith**. A user, an application, or another module writes an Agent Smith—a script that declares an agent's purpose, the scenes in which it may act, and the few standing quantities that make it an agent at all—and the compiler turns that declaration into a running agent. The name is literal: every script *is* an Agent Smith, and the compiler is the smith that forges the agent the script describes.

1.2 What the language is, and is not

Agent Smith is a *specification and instantiation* language. Its scope is deliberately narrow and its boundary is the crux of the design:

- Agent Smith *specifies* an agent: its purpose (what it is for), its scenes (the activities that may serve that purpose), its identity material, its coherence condition, its attention budget, its floors.
- Agent Smith *instantiates* an agent: the compiler reads a well-typed script and produces a running agent that honours the runtime invariants by construction.
- Agent Smith does *not* specify what the agent *does* in a domain. How a blade is forged, how bread is baked, how a task is solved is *not* in the language. That competence is supplied by the specifier through a constructor hook (the declared drive and the per-scene behaviour), exactly as the competence of an instrument is the player's and not the score's. Agent Smith carries the intent; the mechanism is delegated.

The language thereby stays thin. It contains no domain grammar, no forging routine, no baking routine, no solver; it contains only the constructs needed to say *what an agent is for* and to build one that pursues it.

1.3 The underlying theory, used as a black box

Agent Smith is a language *over* a fixed theory of purpose-directed agents, which it does not reprove. That theory supplies, for any agent satisfying a handful of axioms (finiteness, presence, costly individuation, an act floor with bounded attention, phase exclusion): a conserved, nonlocal, positive *identity* invariant; a *purpose* that is the attractor of a strongly convex *drive*; a *water-filling* division of attention across concurrent *scenes* under a single price; a *monotone* committed history under which an agent never re-occupies a past state and an authored copy is a distinct individual; *search-not-fetch* world access, so a reply is walked afresh over the agent's present graph rather than returned from a store; *construction/commitment phase exclusion*; *two-factor relevance* (a response is admissible only if it both advances purpose and preserves coherence); and, one level up, a *society* that is again an agent-shaped object with its own identity and its own single shared purpose, coordinating in a common *outcome space* without any shared internals. Agent Smith's job is to let a specifier *name* the free parts of such an agent—its drive, its scenes, its material—and to *compile* the naming into an instance for which all of the above holds. We treat the theory's results as given and cite them where each is consumed; a reader who grants them can check every claim about the language.

1.4 Mechanism agnosticism, stated once

The single most important property of Agent Smith is that it is *mechanism-agnostic*. The specifier says the agent is *for* forging (a drive

whose attractor is "blades exist, to specification") and that forging, contracting, and sourcing-ore are *scenes* it may act in; the specifier does *not*, in Agent Smith, write how forging is performed. That is supplied as a hook. Consequently the same language, with different declared purposes and scenes, specifies a game smith and a task-solving agent with no change to the language: by the underlying theory the two are one object—an agent pursuing an attractor—differing only in whether the attractor's cell is *standing* (never saturated, so the agent runs on: a character with a life) or *attainable* (reached, so the agent halts: a task done). Agent Smith instantiates both from the same constructs.

1.5 Contributions

1. A grammar for Agent Smith (Section 3): the constructs by which a user, application, or module specifies an agent by its purpose and scenes.

2. A denotation (Section 4): each construct is given a meaning as an object of the underlying agent theory—self-graph, drive, scene, coherence, budget, society.

3. A type system (Section 5) that admits exactly the specifications the compiler can instantiate into an agent honouring the invariants, and a proof that typing is decidable.

4. A lowering semantics (Section 6) taking a well-typed script to an agent instance, and the *Instantiation Theorem* (Section 7): every well-typed Agent Smith compiles to an agent that honours the four runtime invariants by construction, and no ill-typed script is instantiated.

5. The *tick-loop* (**??**): a small-step specification of what a compiled agent does—observe the solution-state (tiered, cheap-first), diagnose whether it is stalled, and commit through a runtime-owned sufficiency gate that reads the outcome, not the agent's reasoning—with a proof that the loop preserves the four invariants and that a pure observer (which commits nothing) is a valid, almost-free member.

6. The mechanism-agnosticism theorem (Section 8): the language carries no domain competence; behaviour is delegated to a constructor hook, and the same script shape instantiates a character or a task-agent.

7. Society as an Agent Smith (Section 10): a configuration of agents is specified by the same language one level up, closed under composition.

1.6 Organisation

Section 2 fixes the agent objects Agent Smith targets, imported from the underlying theory. Section 3 gives the grammar. Section 4 gives the denotation. Section 5 gives the type system. Section 6 gives the lowering semantics. **??** specifies the tick-loop—what a compiled agent does each tick (observe, diagnose, commit), with the pure observer as its floor case. Section 7 proves the Instantiation Theorem. Section 8 proves mechanism agnosticism. Section 9 covers management (spawn, snapshot, retire). Section 10 lifts to societies. Section 11 gives worked Agent Smiths—a smith and a task-agent. Section 12 states scope.

2 The agent objects Agent Smith targets

Agent Smith compiles specifications into *agent objects*. We fix those objects here, importing them from the underlying theory of purpose-directed agents; each is used by the compiler as a black box with the stated guarantee. Nothing in this section is proved anew.

Definition 2.1 (Agent object)**.** An *agent object* is a tuple

$$\mathsf{A} = (\Gamma,\ \Phi,\ \mathcal{S},\ \kappa,\ \alpha,\ \beta_a,\ m)$$

where: $\Gamma = (V, E, w)$ is a finite connected weighted *self-graph* with weight floor $\beta > 0$ (the internal distinctions and the costs of holding them apart); Φ is a strongly convex *drive potential* on a compact convex disposition space $\mathcal{B} \subseteq \mathbb{R}^n$, with unique minimiser the *purpose* $x^\star = \arg\min \Phi$; $\mathcal{S}$ is a finite family of *scenes*, each a pair $(A_\mathsf{S}, \gamma_\mathsf{S})$ of an outcome map $A_\mathsf{S} : \mathcal{B} \to \mathcal{O}$ into a common outcome space $\mathcal{O}$ and a gain profile γ_S; κ is a *coherence* predicate

on candidate responses; $\alpha > 0$ is the *attention budget*; $\beta_a > 0$ is the *act floor*; and $m \in \mathbb{N}$ is the *committed count*.

Definition 2.2 (The four runtime invariants)**.** An agent object, once running, is required to preserve four *invariants*:

(I1) *Conserved identity.* The character invariant $\chi(\mathsf{A}) = \min_{\mathcal{Q}:k\geq 2} \sum_{i<j} w(\partial(U_i, U_j))$ is strictly positive, non-local, and unchanged under every relabelling of the parts.

(I2) *Never-resetting count.* m is incremented on every committed act and never decremented.

(I3) *Search-not-fetch.* Every response is the terminus of a fresh residual-descending walk on the present Γ; no answer is returned from a store without a walk.

(I4) *Phase exclusion.* At each instant the agent is in exactly one of a construction phase (updating Γ, committing no act) or a commitment phase (committing an act, forming no distinction).

Principle 2.3 (The theory guarantees the invariants for any agent object)**.** For any agent object A (Definition 2.1) whose self-graph has a positive floor, whose drive is strongly convex, whose scenes have the stated form, and whose acts are floored, the underlying theory guarantees: (I1) identity is a conserved positive non-local region; (I2) the committed count is strictly monotone and copies are distinct; (I3) recalling is searching; (I4) construction and commitment alternate; and, additionally, attention over the scenes divides by a single-price water-filling rule, a response is admissible iff it is two-factor relevant (purpose gain and coherence margin), and a society of agent objects is again an agent object with its own identity and single shared purpose, coordinating in $\mathcal{O}$ without shared internals. We take these as given.

Remark 2.4 (Agent Smith's whole job)**.** Principle 2.3 says the invariants hold for *any* well-formed agent object. Agent Smith's entire responsibility is therefore to guarantee that what it compiles *is* a well-formed agent object: a positive-floor self-graph, a strongly convex drive, well-formed scenes, a floored act, a coherence predicate, a positive budget. The type system (Section 5) is exactly the decidable check that a script denotes such an object; the Instantiation Theorem (Section 7) is that a well-typed script always does. The compiler does not re-prove the invariants; it earns them by producing an object the theory already covers.

3 Grammar: how one specifies an agent

An Agent Smith is a script. We give its abstract grammar and then its concrete syntax. The grammar has exactly the constructs needed to name the free parts of an agent object (Definition 2.1) and no others; in particular it has no construct for a domain routine.

3.1 Abstract grammar

Definition 3.1 (Agent Smith, abstractly)**.** An *Agent Smith* $\mathtt{s}$ is a term of the grammar

$$
\begin{array}{lcl}
\mathtt{s} & ::= & \mathtt{agent}\ \iota\ \{\ \delta\ \sigma\ \rho\ \} \ \mid\ \mathtt{society}\ \iota\ \{\ \overline{\mathtt{s}}\ \theta\ \} \\
\delta & ::= & \mathtt{purpose}\ \pi \quad \text{(the drive / attractor)} \\
\sigma & ::= & \mathtt{scenes}\ \{\ \overline{S}\ \} \\
S & ::= & \mathtt{scene}\ \iota\ \mathtt{serves}\ \pi\ \mathtt{with}\ h \\
\rho & ::= & \mathtt{self}\ \{\overline{p},\ \overline{e}\}\ \mathtt{budget}\ \alpha\ \ \mathtt{floor}\ \beta\ \ \mathtt{coherence}\ \kappa \\
\pi & ::= & \mathtt{minimise}\ \phi \ \mid\ \mathtt{reach}\ o \in \mathcal{O} \\
\theta & ::= & \overline{\mathtt{tie}}\ \ \mathtt{couple}\ K
\end{array}
$$

where ι is an identifier; ϕ names a strongly convex potential and o an outcome; $\overline{p}, \overline{e}$ are declared parts and separations; h is a *behaviour hook* (the delegated mechanism); $\overline{\mathtt{tie}}$ are inter-agent separations; K is a coupling strength. Overlines denote finite lists.

Remark 3.2 (There is no rule for "what the scene does"—only that it serves the purpose)**.** The scene form is $\mathtt{scene}\ \iota\ \mathtt{serves}\ \pi\ \mathtt{with}\ h$. It names the scene, declares *which purpose it serves*, and binds a hook h that performs the domain work. The grammar has no production expanding h into forging steps or baking steps: h is opaque to Agent Smith, supplied by the specifier or a module. This is the grammatical form of mechanism agnosticism (Section 8): the

language can say a scene serves a purpose, never how it serves it.

3.2 Concrete syntax

The concrete syntax is a lightweight surface over Definition 3.1. A specification of a smith reads:

```
// An Agent Smith: the iron smith.
// Says what the smith IS FOR; not how it
    forges.
agent smith {
  purpose minimise forge_residual
    // attractor: blades exist, to spec

  scenes {
    scene hammer   serves forge_residual with
    forge_hook
    scene contract serves forge_residual with
    contract_hook
    scene source   serves forge_residual with
    source_ore_hook
  }

  self {
    parts { skill, stock, orders, reputation }
    separations {
      (skill, stock: 3), (stock, orders: 2),
      (orders, reputation: 2), (reputation,
    skill: 2)
    }
  }
  budget 1.0
  floor  2.0
  coherence keeps { reputation, skill }
}
```

The three scenes—`hammer`, `contract`, `source`—are the smith's "life": all serve the one purpose `forge_residual`, and which the smith does at a moment is not scripted but decided by water-filling over their gains (Principle 2.3). The hooks `forge_hook`, `contract_hook`, `source_ore_hook` carry the domain competence and are not part of Agent Smith.

Remark 3.3 (The specifier writes intent, the compiler writes the agent)**.** Everything in the script above is *intent*: what the smith is for, which activities serve it, what holds it together, how much attention it has. Nowhere does the script say "at tick 5, hammer." The compiler (Section 6) turns the intent into an agent object whose conduct is the pursuit of `forge_residual` through the declared scenes, and the theory (Principle 2.3) supplies the conduct.

4 Denotation: what each construct means

We give each construct a denotation as a piece of an agent object (Definition 2.1). The denotation map $[\![\cdot]\!]$ is the specification of the compiler's front end: it says what object a script is asking for, before typing decides whether the request is admissible.

Definition 4.1 (Denotation)**.** The denotation $[\![\mathsf{s}]\!]$ of an Agent Smith is defined by structural recursion:

$$
\begin{aligned}
[\![\texttt{self}\{\overline{p},\overline{e}\}]\!] &= \Gamma = (V,E,w),\ V=\overline{p},\ E=\overline{e} \\
& \quad w(e) = \text{declared cost of } e; \\
[\![\texttt{purpose minimise}\ \phi]\!] &= \Phi := \phi,\ x^\star := \arg\min \phi; \\
[\![\texttt{purpose reach}\ o]\!] &= \Phi := \tfrac{1}{2}\, d(A(\cdot),o)^2,\ x^\star := A^{-1} \\
[\![\texttt{scene}\ \iota\ \texttt{serves}\ \pi\ \texttt{with}\ h]\!] &= \mathsf{S} = (A_{\mathsf{S}}, \gamma_{\mathsf{S}}),\ \text{hook}(h); \\
[\![\texttt{budget}\ \alpha]\!] &= \alpha; \quad [\![\texttt{floor}\ \beta]\!] = \beta; \quad [\![\texttt{cohe} \\
[\![\texttt{agent}\ \iota\{\delta\sigma\rho\}]\!] &= \mathsf{A} = (\Gamma,\Phi,\mathcal{S},\kappa,\alpha,\beta_a,0); \\
[\![\texttt{society}\ \iota\{\overline{\mathsf{s}}\,\theta\}]\!] &= \Sigma = \text{quotient of } [\![\overline{\mathsf{s}}]\!] \text{ under } \theta.
\end{aligned}
$$

The committed count of a freshly denoted agent is 0 (I2's base).

Remark 4.2 (Two ways to name a purpose)**.** Definition 4.1 gives two forms of `purpose`. `minimise` ϕ names the drive directly by a strongly convex potential—the general case. `reach` o names an outcome $o \in \mathcal{O}$ and synthesises the drive as squared distance to it—the common case, and the one that makes a *task*-agent: its attractor is the point o, attainable, so it halts at quiescence. `minimise` with a floored potential whose minimum is a standing region makes a *character*: never saturated, so it runs on. The same construct, two regimes (Theorem 8.3).

5 Type system: which specifications are instantiable

Not every script denotes a *well-formed* agent object. A drive that is not strongly convex has no unique purpose; a self-graph with a zero-weight separation has no floor; a scene serving no declared purpose is dead; a non-positive budget admits no attention. The type system admits exactly the scripts whose denotation

is a well-formed agent object, so the compiler instantiates all and only those.

Definition 5.1 (Judgements)**.** The typing judgement $\vdash \mathsf{s} : \mathsf{Agent}$ reads "s is a well-formed Agent Smith and denotes a well-formed agent object." Auxiliary judgements type the parts: $\vdash \rho : \mathsf{Self}$, $\vdash \delta : \mathsf{Drive}$, $\vdash S : \mathsf{Scene}(\pi)$ ("scene S serves purpose π").

Rule 5.2 (Self well-formed)**.** $\vdash$ `self`$\{\overline{p}, \overline{e}\}$ `floor` β : Self holds iff the graph $(\overline{p}, \overline{e})$ is connected, simple, nonempty, and every declared cost is $\geq \beta > 0$. (Positive floor and connectedness: the hypotheses of the identity theorem.)

Rule 5.3 (Drive well-formed)**.** $\vdash$ `purpose` π : Drive holds iff π denotes a strongly convex, continuously differentiable potential on a compact convex disposition space (for `minimise` ϕ, that ϕ is so; for `reach` o, automatic, as squared distance is 1-strongly convex). (Strong convexity: the hypothesis of unique purpose and attraction.)

Rule 5.4 (Scene serves a declared purpose)**.** $\vdash$ `scene` ι `serves` π `with` $h : \mathsf{Scene}(\pi)$ holds iff π is the agent's declared purpose, the gain profile γ_{S} is nondecreasing with $\gamma_{\mathsf{S}}(0) = 0$, and the hook h has the outcome-map type $\mathcal{B} \to \mathcal{O}$. A scene serving no declared purpose does not type. (No dead scenes: every scene must reduce the one residual.)

Rule 5.5 (Agent well-formed)**.**

$$\frac{\vdash \rho : \mathsf{Self} \quad \vdash \delta : \mathsf{Drive} \quad (\vdash S_i : \mathsf{Scene}(\pi))_i \quad \alpha > 0 \quad \beta_a > 0}{\vdash \texttt{agent}\ \iota\{\delta\ \texttt{scenes}\{\overline{S}\}\ \rho\} : \mathsf{Agent}}$$

All parts well-formed, all scenes serving the one declared purpose, budget and act floor positive.

Theorem 5.6 (Typing is decidable)**.** *Whether* $\vdash \mathsf{s} : \mathsf{Agent}$ *is decidable in time polynomial in the size of* s*, given oracles for (a) connectivity of a finite graph and (b) strong convexity of the named potential.*

Proof. Rule 5.2 checks connectivity (linear in $|E|$), simplicity, nonemptiness, and a floor comparison per edge—all polynomial. Rule 5.3 defers to the strong-convexity oracle; for `reach`, it is immediate. Rule 5.4 checks a declared-purpose match (identifier equality), monotonicity and the zero of the profile (given the profile's form), and a hook type. Rule 5.5 is a finite conjunction of the above plus two positivity checks. Each premise is polynomial and there are finitely many, so the judgement is decidable in polynomial time. □

Remark 5.7 (The type system is exactly the well-formed-object check)**.** Rules 5.2 to 5.5 are, one for one, the hypotheses Principle 2.3 needs: positive floor and connectedness (identity), strong convexity (purpose), well-formed scenes (water-filling), positive act floor (monotone count and search-not-fetch). Typing is not an extra discipline layered on the theory; it *is* the theory's hypotheses made into a decidable front-end check.

6 Lowering: instantiation as compilation

The compiler $\mathcal{C}$ takes a well-typed Agent Smith to a running agent instance by a small-step lowering relation $\rightsquigarrow$. Lowering is where "specification" becomes "instance."

Definition 6.1 (Lowering)**.** The lowering relation $\mathsf{s} \rightsquigarrow \mathsf{A}$ is defined for $\vdash \mathsf{s} : \mathsf{Agent}$ by:

L1. *Realise the self-graph.* Choose a concrete labelling of $[\![\rho]\!]$; the identity invariant is label-independent (I1), so any labelling is admissible.

L2. *Bind the drive.* Install $\Phi = [\![\delta]\!]$ with its minimiser $x^\star$; by Rule 5.3 it is strongly convex, so $x^\star$ exists and attracts.

L3. *Install the scenes.* For each typed scene, register $(A_{\mathsf{S}}, \gamma_{\mathsf{S}})$ and bind its hook h; the compiler holds h as an opaque callable (Section 8).

L4. *Set the standing quantities.* α, β_a, κ from ρ; the committed count $m := 0$.

L5. *Seat the runtime.* Attach the water-filling scheduler over the scenes, the two-factor relevance gate, the phase alternator, and the search-walk responder—the fixed runtime of Principle 2.3.

The result is an agent object A with $m = 0$, ready to run.

Remark 6.2 (The compiler installs the runtime; it does not write conduct)**.** Step L5 attaches a *fixed* runtime—the same scheduler, gate, alternator, and responder for every agent, supplied once by the theory. The compiler writes no conduct: the agent's behaviour is what that fixed runtime produces when driven by *this* agent's drive and scenes. Two different Agent Smiths differ only in the objects installed at L1–L4; the machinery at L5 is identical. This is why the language can be thin and the agents various.

7 The tick-loop: what a compiled agent does

Lowering (Section 6) seats a fixed runtime at L5. We now specify that runtime as a small-step relation—the *tick-loop*—because it is the one piece of an agent's behaviour that is common to every Agent Smith and is worth pinning exactly. The loop realises the framework's governing stance: *an agent acts on the state of the solution, not on its own internal reasoning.* Each tick, an agent reads the solution-state, diagnoses whether it is stuck, and either commits an act or stays silent; nowhere does it introspect a stored model of the whole task. The loop has three steps—*observe*, *diagnose*, *commit*—and a distinguished floor case, the pure observer, that commits nothing and costs almost nothing.

7.1 The solution-state and its tiered reading

Definition 7.1 (Solution-state; residual gap)**.** The *solution-state* Ω is the shared, external record of the work done so far toward the intent—the parts individuated, the boundaries committed. It is *not stored inside any agent*: no agent holds Ω; each reads a slice of it on demand. For an agent A with purpose $x^\star$, the *residual gap* $\Delta_{\mathsf{A}}(\Omega) \geq 0$ is the semantic distance from the current solution-state to A's attractor, floored (by the underlying theory) at $\Delta_{\mathsf{A}} \geq \beta > 0$: the gap never closes to a point.

Definition 7.2 (Reading tiers)**.** A *reading* of Ω is tiered by cost:

- *Tier 0 (probe).* A deterministic slice retriever $\text{probe}(\Omega, q)$: given a query q (what this agent needs), return a ranked context slice of the present Ω, cheaply and with no stored answer—a re-derivation from Ω each call. (Concretely, an empty-dictionary index over the substrate that returns a ranked slice per query; it holds no domain answers, only the means to search for them.)
- *Tier 1 (deep read).* A model-backed extraction invoked only when a Tier-0 slice does not suffice to move the residual—the escalation.

The agent always reads Tier 0 first and escalates to Tier 1 only on residue (**??**).

Remark 7.3 (Tier 0 is search-not-fetch, already)**.** The probe of **??** is exactly the search-not-fetch discipline (I3) made concrete: it returns a slice re-derived from the current Ω, never a stored answer, so a repeated probe against a grown Ω returns a different slice. An agent that only ever reads Tier 0 therefore never fetches and never holds the task; it re-reads the state. This is why the framework's persistence is in the *trajectory*—how the agent got here, its monotone count and its position in the work—not in a cached task (Remark 9.3).

7.2 The three steps

Definition 7.4 (Observe)**.** *Observe* reads the solution-state for this agent's purpose:

O1. probe Tier 0: $\varsigma \leftarrow \text{probe}(\Omega, q_{\mathsf{A}})$;

O2. measure the residual gap $\Delta_{\mathsf{A}}(\varsigma)$ from the slice;

O3. if the slice suffices to determine the next act (the gap is legible from Tier 0), stop; else escalate to Tier 1 and re-measure.

Observe returns the current residual gap and a candidate next act, computed from the solution-state slice alone—never from a stored model of the whole task.

Definition 7.5 (Diagnose)**.** *Diagnose* reads the *descent* of the residual gap across recent ticks and returns a typed limit:

- CONVERGING: the gap fell on the last committed act (Δ strictly decreased)—compute is the only obstacle; keep acting.

- COMPUTE-LIMITED: the gap is falling slowly but steadily—feed more attention.

- STRUCTURE-LIMITED: the gap has not fallen over a window (a *stall*) though acts were committed—more of the same act will not help; the agent emits the stall as a typed signal so the society may route a *different* agent to the gap.

A stalled agent does not fall silent: it emits STRUCTURE-LIMITED, a diagnosis the conductor layer reads.

Definition 7.6 (Commit; the sufficiency gate)**.** *Commit* is a *runtime-owned* gate—not part of the agent's domain work. It reads the *outcome* of the candidate act (its effect on Ω in the common outcome space) and fires iff two conditions hold:

(i) *sufficiency*: the candidate act lowers the residual gap of the *parent* intent—its outcome-gain clears the attention price (*not* that the act reduces the agent's own gap to its floor: release at sufficiency, not at completion);

(ii) *coherence*: the act preserves the agent's coherence condition (two-factor relevance).

If both hold, the agent commits: it deposits an act, increments the count m (I2), and changes Ω—which every other agent will read next tick. If either fails, the agent commits nothing this tick and remains an observer.

Remark 7.7 (The gate reads the outcome, not the reasoning)**.** **??** is the formal seat of "act on the state of the solution, not on internal reasoning." The commit decision is a function of the act's *outcome* in the shared space—does it move the parent's residual, does it stay coherent—and never of how the agent's hook computed the act. The domain work is opaque (Section 8); the decision to act is the runtime's, read off the outcome. This is why an agent needs no model of any other agent: it reads the shared solution-state, not anyone's internals (coordination in outcome space, Principle 2.3).

7.3 The loop, and its floor case

Definition 7.8 (Tick-loop)**.** The *tick* of a compiled agent A against solution-state Ω is

$$\mathrm{tick}(\mathsf{A}, \Omega) \;=\; \mathrm{commit} \circ \mathrm{diagnose} \circ \mathrm{observe}\,,$$

in the construction/commitment phase order of I4: observe and diagnose run in a construction phase (no act), commit runs in a commitment phase (at most one act). The agent's behaviour is the iteration of tick against the evolving Ω.

Theorem 7.9 (The tick-loop preserves the four invariants)**.** *Iterating* tick *(**??**) preserves I1–I4. Identity is untouched by any step (observe and diagnose read Ω; commit changes Ω, not A's self-graph partition), so I1 holds; commit increments m and no step decrements it, so I2 holds; observe reads only via the probe, which re-derives from the present Ω with no stored answer, so I3 holds; and observe/diagnose occupy a construction phase while commit occupies a commitment phase, disjoint by **??**, so I4 holds.*

Proof. By **????**, observe and diagnose compute from a slice of Ω and the residual descent; they commit no act and do not alter $\Gamma = (V, E, w)$, so the character invariant $\chi(\mathsf{A})$ (Definition 2.2) is unchanged (I1) and the count is untouched. By **??**, commit deposits at most one act, incrementing m by one; no step lowers m (I2). By **??** the probe returns a re-derived slice, never a stored answer, and Tier 1 is a model extraction against the present state, so no response is fetched (I3). By **??** observe/diagnose are in construction and commit is in commitment, and I4's phase exclusion makes these disjoint instants, so no tick both constructs and commits (I4). Hence all four are preserved by every tick, and by induction by the whole run. □

Definition 7.10 (Pure observer; the floor case)**.** An agent is a *pure observer* at a tick if its commit gate does not fire: either the candidate act's outcome-gain does not clear the price (the solution-state does not yet call for it) or it would break coherence. A pure observer runs observe (Tier 0 only—it never escalates, having no act to justify the cost) and diagnose, commits nothing, and leaves Ω and its own count m unchanged.

Theorem 7.11 (The pure observer is valid and almost free)**.** *A pure observer is a well-formed*

member of the society: it honours I1–I4 vacuously (it commits no act, so I2's count is unchanged and I4's commitment phase is empty), it holds no task (I3: it only ever probes Tier 0), and it costs only its Tier-0 probe per tick. It contributes nothing to the residual ($\delta = 0$) yet remains present: the instant the solution-state changes so that its candidate act clears the price, its commit gate fires and it acts. A society may contain arbitrarily many pure observers at negligible cost.

Proof. A pure observer runs observe and diagnose (construction phase, no act) and a commit gate that does not fire (**??**); so it deposits no act, leaving m fixed (I2 vacuous) and the commitment phase empty (I4 vacuous), while I1 (identity) and I3 (Tier-0 probe, re-derived slices) hold as for any agent (**??**). Its per-tick cost is one Tier-0 probe, by **??**. Because the commit gate is re-evaluated each tick against the current Ω (**??**), a change in the solution-state that raises the observer's candidate outcome-gain above the price fires the gate on that tick: the observer acts exactly when the state calls for it, not before. Hence it is present, valid, and almost free. □

Remark 7.12 (The three at the zoo, and the crowd of cheap readers)**.** **??** is the penguin-watcher: an agent attending to its own scene, holding no model of the others, that commits the instant the shared outcome-state changes (the others flee) because at that tick its "get to safety" act clears the price—coordination through the outcome, not through anyone's internals. The same construction explains why a society prefers *many cheap readers* to one expensive one where a judgment is uncertain: independent observers reading the same outcome-state sharpen the collective judgment multiplicatively (Principle 2.3, crowd sharpening), and each costs only its Tier-0 probe. The loop makes the cheap-crowd the default and the deep read (Tier 1) the exception, paid only on escalation. We use only the loop's step relation; the readings are orientation.

8 The Instantiation Theorem

We prove the compiler's central guarantee: well-typed scripts instantiate invariant-honouring agents, and ill-typed scripts do not instantiate at all.

Theorem 8.1 (Instantiation soundness)**.** *If* $\vdash \mathsf{s} : \mathsf{Agent}$ *then* $\mathsf{s} \rightsquigarrow \mathsf{A}$ *for a unique agent object* A *up to relabelling, and* A *honours the four runtime invariants* (I1–I4) *of Definition 2.2 by construction.*

Proof. By Rule 5.5, a well-typed s has a well-formed self (Rule 5.2: connected, positive floor), a well-formed drive (Rule 5.3: strongly convex), scenes each serving the declared purpose (Rule 5.4), and positive α, β_a. Its denotation $[\![\mathsf{s}]\!]$ (Definition 4.1) is therefore a well-formed agent object (Definition 2.1). Lowering (Definition 6.1) realises it: L1–L4 produce exactly that object, and L5 attaches the fixed runtime. By Principle 2.3, every well-formed agent object honours I1 (identity: positive floor + connectedness give a conserved non-local positive invariant), I2 (act floor gives a strictly monotone count), I3 (search-not-fetch: a floored act makes a zero-act fetch return nothing, so every answer is a walk), and I4 (phase exclusion is part of the fixed runtime). Uniqueness up to relabelling: L1's only freedom is the labelling, under which the object is determined and, by I1, identity-invariant. Hence A is unique up to relabelling and honours I1–I4. □

Theorem 8.2 (No ill-typed instantiation)**.** *If* $\nvdash \mathsf{s} : \mathsf{Agent}$ *then* $\mathcal{C}$ *emits no agent: the compiler rejects* s *at typing (Theorem 5.6) before lowering. In particular no agent is ever instantiated with a zero-floor self, a non-convex drive, a dead scene, or a non-positive budget.*

Proof. $\mathcal{C}$ runs the decidable type check (Theorem 5.6) before lowering (Definition 6.1) and lowers only on $\vdash \mathsf{s} : \mathsf{Agent}$. An ill-typed script fails some premise of Rules 5.2 to 5.5—the four failure modes named—so it is rejected and never reaches $\rightsquigarrow$. Hence no agent object violating the theory's hypotheses is ever produced. □

Corollary 8.3 (The compiler is the sole instantiator)**.** *Every agent in the system is the image under* $\mathcal{C}$ *of a well-typed Agent Smith. There is no other route to an agent object, so every agent in the system honours I1–I4 (Theorem 7.1) and none violates the hypotheses (Theorem 7.2). Instantiation is centralised in the compiler.*

Proof. Agent objects arise only by lowering (Definition 6.1), which $\mathcal{C}$ gates on typing. Combine Theorems 7.1 and 7.2. □

9 Mechanism agnosticism

We prove the language's defining property: it carries no domain competence. The behaviour of a scene is a constructor-supplied hook the compiler treats as opaque, so the same language instantiates a game character and a task-agent without change.

Definition 9.1 (Behaviour hook)**.** A *behaviour hook* h bound to a scene is a callable of type $\mathcal{B} \times \text{Interaction} \to \mathcal{B} \times \mathcal{O}$: given the agent's disposition and an interaction, it returns the next disposition and an outcome. Agent Smith holds h as an opaque value: no production of Definition 3.1 inspects or expands h, and no typing rule constrains h beyond its type (Rule 5.4).

Theorem 9.2 (The language contains no domain competence)**.** *The denotation and typing of an Agent Smith are independent of the internal behaviour of any hook: for two scripts* s, s' *identical except that each bound hook* h_i *is replaced by a hook* h_i' *of the same type, we have* $\vdash \mathsf{s} : \mathsf{Agent} \iff \vdash \mathsf{s}' : \mathsf{Agent}$*, and* $[\![\mathsf{s}]\!], [\![\mathsf{s}']\!]$ *differ only in the installed callables, not in* $\Gamma, \Phi, \alpha, \beta_a, \kappa$*. Hence Agent Smith specifies* that *a scene serves the purpose, never* how*.*

Proof. Typing (Rules 5.2 to 5.5) constrains a hook only through its type $\mathcal{B} \to \mathcal{O}$ (Rule 5.4); no rule reads h's body. Replacing h_i by same-typed h_i' leaves every premise unchanged, so the judgements coincide. Denotation (Definition 4.1) records the hook as $\text{hook}(h)$, an opaque binding; the self-graph, drive, budget, floor, and coherence are computed from the other constructs and are untouched by the swap. Hence the objects differ only in installed callables. □

Theorem 9.3 (One language, character or task-agent)**.** *Let* s_{char} *and* s_{task} *be Agent Smiths identical except in their* `purpose`*:* s_{char} *uses* `minimise` ϕ *with* ϕ*'s minimum a standing region (never saturated), and* s_{task} *uses* `reach` o *with* o *an attainable outcome. Both type by the same rules and lower by the same relation; the instantiated agents differ only in whether the attractor's cell is standing or attained. Thus the same language, unchanged, instantiates a character that lives on and a task-agent that halts.*

Proof. Both `minimise` ϕ (with ϕ strongly convex) and `reach` o satisfy Rule 5.3 (Remark 4.2), so both scripts type by Rule 5.5 and lower by Definition 6.1. By Principle 2.3 the runtime is identical; the only difference is the drive installed at L2, hence the location of $x^\star$. A standing-region minimum is never saturated (the floor keeps residual positive), so the agent runs on; an attainable o is reached at quiescence, so the agent halts. No construct other than `purpose` differs. Hence one language instantiates both regimes. □

Remark 9.4 (Why this is the whole point)**.** Theorems 8.2 and 8.3 are the reason Agent Smith is a single small language rather than a family of domain languages. Because it holds mechanism as an opaque hook and purpose as the only thing distinguishing a character from a task, a user specifies "an iron smith" and "an agent that solves this task" with the same constructs, and the compiler instantiates each as an agent pursuing its declared purpose through its declared scenes. The language never learns to forge or to solve; it learns only to build an agent that is *for* forging or solving. The interpretation firewall of the underlying theory is thereby inherited by the language: Agent Smith speaks only of purpose, scenes, identity, budget—never of a domain.

10 Management: spawn, snapshot, retire

Agent Smith specifies *and manages*. Beyond instantiation, the language provides the lifecycle operations, each with a meaning that preserves the invariants.

Definition 10.1 (Lifecycle operations)**.** For a compiled agent A:

- `spawn` s: run $\mathcal{C}(\mathsf{s})$ and register the resulting A at count 0.
- `snapshot` A: emit a serialisable record $(\Gamma, \alpha, \beta_a, \kappa, \text{disposition}, m, \text{phase})$—

everything but the opaque hooks, which are re-bound on rehydration.

- `configure` A: adjust budget α or coupling within the running agent *in the construction phase only* (I4).

- `retire` A: stop the agent; its snapshot persists, so a later `spawn` from that snapshot resumes the *same* individual at its accumulated count (not a reset).

Theorem 10.2 (Lifecycle preserves the invariants)**.** *Each operation of Definition 9.1 preserves I1–I4. In particular* `snapshot/spawn` *round-trips the committed count, so a rehydrated agent resumes at its accumulated m and is the same individual; and* `configure` *runs only in construction, so it never violates phase exclusion.*

Proof. `spawn` is $\mathcal{C}$, covered by Theorem 7.1 (I1–I4 hold at birth). `snapshot` records m and the label-independent self-graph; `spawn`-from-snapshot restores them, so m is preserved (I2) and identity is the same invariant (I1). Because the count is carried, the rehydrated agent is not a fresh copy at 0 but the same individual at m (monotone-count distinctness of the underlying theory). `configure` changes only α or coupling and only in the construction phase, so it commits no act (I4 preserved) and does not touch the self-graph's partition structure (I1 preserved) nor the count (I2). `retire` stops dispatch and persists the snapshot; no count is lowered (I2). Search-not-fetch (I3) is a property of the responder, attached at L5 and untouched by lifecycle. Hence all four are preserved. □

Remark 10.3 (Persistence is what makes the smith the same smith tomorrow)**.** Theorem 9.2 is the formal content of "the smith you met yesterday, mid-contract, is still that smith today." Because `snapshot`/`spawn` carries the committed count, resuming an agent is resuming the *same* individual with its accumulated history, not instantiating a look-alike at count zero. A re-run of an agent's past act is therefore a new act at a higher count—a genuine re-derivation against the grown graph, not a replayed cache—which is exactly what a persistent character, or a resumable task, requires.

11 A society is an Agent Smith

The language lifts to configurations of agents by the same constructs, one level up. A `society` is itself an Agent Smith, so any configuration the specifier wants is written in the same language.

Definition 11.1 (Society specification)**.** A `society` ι { $\bar{\mathsf{s}}$ θ } denotes the quotient object Σ whose vertices are the agents $[\![\bar{\mathsf{s}}]\!]$ and whose edges are the declared ties θ, with coupling K for phase synchronisation. Its purpose, if declared, is a society drive on the product disposition space whose minimiser maps to the shared purpose in the common outcome space $\mathcal{O}$.

Theorem 11.2 (Society well-formedness and closure)**.** *If each $\mathsf{s}_i \in \bar{\mathsf{s}}$ is well-typed and every declared tie has cost $\geq \beta$ and the society is connected, then* $\vdash$ `society` $\iota\{\bar{\mathsf{s}}\,\theta\}$: Agent*: a well-formed society is itself a well-formed agent object, honouring I1–I4 at the society level, and coordinating in $\mathcal{O}$ without shared internals. Hence the language is closed under composition: an Agent Smith may be an agent or a society of Agent Smiths.*

Proof. By Principle 2.3 a connected quotient of agent objects with floor-$\geq \beta$ ties is again an agent object (finite, connected, positive floor), so Rule 5.5 applies to Σ: it has a conserved nonlocal identity (I1 at agent level), a monotone society count (I2), search-not-fetch and phase exclusion inherited by each member and by the quotient's own scheduling (I3, I4). Coordination without shared internals is the outcome-space coordination of the theory: members attain a shared region in $\mathcal{O}$ regardless of disjoint self-graphs. The society form is a production of Definition 3.1, so the language is closed under it. □

Remark 11.3 (Any configuration, written once)**.** Theorem 10.2 is what lets a specifier instantiate agents "in whatever configuration": a lone smith, a smith and a miner tied by ore-supply, a whole town, an ensemble of task-agents cooperating on a problem—each is a `society` script, compiled by the same $\mathcal{C}$, honouring the same invariants. Because coordination lives in outcome space, the specifier need give no agent a model of any other; declaring

the ties and the shared outcome suffices. The society's single shared purpose (one drive on the quotient) is the formal sense in which the configuration acts as one.

12 Worked Agent Smiths

We instantiate two agents from the same language: a character and a task-agent, differing only in `purpose`.

12.1 A character: the iron smith

The smith of Section 3, whose purpose `minimise forge_residual` has a standing minimum: there is always more to forge, so the agent runs on. Its three scenes are its life; water-filling picks which it works at each moment; the necessity floor makes it ignore off-purpose requests (a scene that does not serve `forge_residual` never types, and an off-purpose interaction gains below the attention price and is declined). It persists across sessions by `snapshot`/`spawn`.

12.2 A task-agent: rerun an experiment

```
// An Agent Smith: a task-solving agent.
// Same constructs; purpose is an attainable
    outcome.
agent rerun_exp {
  purpose reach verdict_confirmed
    // attractor: the experiment's verdict =
    CONFIRMED

  scenes {
    scene integrate serves verdict_confirmed
    with kuramoto_hook
    scene tabulate  serves verdict_confirmed
    with aggregate_hook
    scene report    serves verdict_confirmed
    with emit_hook
  }
  self {
    parts { data, method, result, verdict }
    separations {
      (data, method: 2), (method, result: 2),
      (result, verdict: 2), (verdict, data: 2)
    }
  }
  budget 1.0
  floor  2.0
  coherence keeps { method, result }
}
```

This types and lowers by the *same* rules as the smith (Theorem 8.3). Its purpose `reach verdict_confirmed` is attainable, so it halts at quiescence when the verdict is confirmed. A `spawn` from a prior snapshot resumes the same task-individual at its accumulated count, so "redo the experiment" re-derives against the grown graph rather than replaying a stored result. The specifier wrote intent; the compiler instantiated an agent that pursues it.

Remark 12.1 (The same smith forges both)**.** The smith and the task-agent are two Agent Smiths, compiled by one compiler, differing only in whether `purpose` names a standing region or an attainable outcome. Everything else—the four invariants, the water-filling over scenes, the persistence, the coordination if placed in a society—is identical. This is the language's economy: to specify a game character and a task-solver is to write the same kind of script, and to instantiate either is one compilation.

13 Scope

Agent Smith is a specification-and-instantiation language and nothing more. It targets the agent objects of a fixed underlying theory, whose runtime guarantees (the four invariants, water-filling attention, two-factor relevance, outcome-space coordination) it consumes as a black box (Principle 2.3) and does not re-prove. Its constructs name the free parts of an agent—purpose, scenes, self, budget, floor, coherence, ties, coupling—and its type system admits exactly the scripts whose denotation is a well-formed agent object (Section 5); its compiler instantiates all and only those (Section 7). The language holds no domain competence: scene behaviour is an opaque constructor hook (Section 8), so the same language instantiates a game character and a task-agent without change (Theorem 8.3), and a configuration of agents is itself an Agent Smith (Section 10). What the language does *not* do is specify how any scene performs its domain work, prescribe an agent's moment-to-moment conduct (that is the runtime's, driven by the declared purpose and scenes), or guarantee anything about a hook's internal correctness (that is the hook author's). One hypothesis is inherited from the

underlying theory and not re-argued here: that an agent is a finite, positively-separated, floored, purpose-directed object; absent it the invariants are not guaranteed and the compiler's soundness does not hold. Interpretive readings—of a script as a character with a life, of a society as a population, of a task-agent as an instantiated task—are confined to remarks on which no theorem depends.

13.1 Conclusion

To specify a non-player character, one says what it is and what it is for and lets its conduct follow; Agent Smith is the language for saying exactly that, and its compiler is the instantiator that turns the saying into an agent. Every script is an Agent Smith: a complete specification of one agent's purpose, the scenes that serve it, and the material that makes it an agent at all. The compiler types the specification against the hypotheses the underlying theory requires, and lowers it into a running agent that honours, by construction, a conserved identity, a never-resetting count, search-not-fetch access, and phase exclusion—so that a smith is the same smith tomorrow, an off-purpose request is ignored because attention costs something, a copy is a new individual, and a re-run genuinely re-derives. The language holds no domain competence: it says a scene serves the purpose, never how, so the same constructs specify a smith who forges and an agent that solves, a lone character and a whole society. A user, an application, or a module specifies the agent the way a designer specifies an NPC; the compiler forges it. That is the whole of Agent Smith: intent in, agent out, invariants kept.